

Отв еты на вопросы к экзамену по АОИС (темы 1–5)

Курс: «Аппаратное обеспечение интеллектуальных систем», специальность 1-40 03 01 «Искусственный интеллект».
Часть 1. Организация и функционирование традиционных интеллектуальных компьютеров.

Ответы составлены **строго по материалу лекций**. Нумерация тем экзамена соотносится с темами лекций так: вопрос 1 → лекц. Тема 1; вопрос 2 → Темы 2 и 4; вопрос 3 → Темы 5 и 6; вопрос 4 → Темы 7 и 8; вопрос 5 → Тема 9.

Тема 1. Этапы развития и поколения ЭВМ. Эволюция их интеллектуальных свойств

Этапы развития и поколения ЭВМ. Определяющие свойства поколения

Традиционно этапы развития ЭВМ классифицируют **по поколениям**. Поколения характеризуются тремя группами показателей:

- **внутренняя организация** — архитектура, программное обеспечение;
- **средства взаимодействия пользователя с ЭВМ** — языки и формы общения;
- **техническая реализация** — элементная база, технические параметры.

Развитие этих показателей шло неравномерно, поэтому деление на поколения **условно** (иногда говорят даже о промежуточных поколениях). Ошибочно отдавать предпочтение какому-либо одному показателю (например, только элементной базе).

Характерные типы ЭВМ поколений и их основные характеристики

Поколение (годы)	Элементная база	Производительность ЦП	Тип и объём ОП	Характерные типы ЭВМ
1-е (1946–1958)	Электронные лампы, реле	До $3 \cdot 10^5$ оп/с	Триггеры, ферритовые сердечники; до 64 Кбайт	Большие
2-е (1959–1963)	Транзисторы, параметроны	До $3 \cdot 10^6$ оп/с	Миниатюрные ферритовые сердечники; до 512 Кбайт	Малые, средние, большие, специальные
3-е (1964–1976)	Интегральные схемы, БИС	До $3 \cdot 10^7$ оп/с	Полупроводниковые БИС; до 16 Мбайт	Большие, средние, мини- и микро-ЭВМ, специальные, ряды ЭВМ
4-е (1977–...)	СБИС	Более $3 \cdot 10^7$ оп/с	Полупроводниковые СБИС; более 16 Мбайт	Супер-ЭВМ, ПК, специальные, общие

Типичные модели и характерное ПО:

Поколение	Типичные модели	Характерное ПО
1-е	EDSAC, ENIAC, UNIVAC, БЭСМ, Минск-1...14, УРАЛ	Коды, автокоды, ассемблеры
2-е	RCA-501, IBM 7090, БЭСМ-6, Минск-2/22/23/32	Языки программирования, диспетчеры, АСУ, АСУТП
3-е	IBM/360, IBM/370, PDP, VAX, ЕС ЭВМ, СМ ЭВМ	ППП, СУБД, САПР, ЯВУ, операционные системы
4-е	IBM/390, SX-2, IBM PC/XT/AT, PS/2, Cray, сети, модели «СКИФ»	БЗ, ЭС, системы параллельного программирования

Сокращения: АСУ — автоматизированная система управления; АСУТП — АСУ технологическим процессом; ППП — пакеты прикладных программ; СУБД — система управления базами данных; САПР — система автоматического проектирования; ЯВУ — язык высокого уровня; БЗ — база знаний; ЭС — экспертные системы.

Типы ЭВМ (большие, малые, микро- и др.) отличаются размерами и вычислительной мощностью.

Классификация персональных компьютеров по поколениям и конструктивному исполнению

ПК классифицируют:

- по поколениям;
- по конструктивным особенностям.

По конструктивному исполнению ПК делятся на:

- стационарные ПК;
- переносные — портативные ПК, ПК-блокноты, карманные ПК, электронные секретари, электронные записные книжки.

По поколениям ПК отличаются разрядностью используемых микропроцессоров:

- ПК 1-го поколения — 8-битовые микропроцессоры;
- ПК 2-го поколения — 16-битовые;
- ПК 3-го поколения — 32-битовые;
- ПК 4-го поколения — 64-битовые.

Эволюция технологии использования ЭВМ

Интеллектуализация ЭВМ — **устойчивая тенденция** развития средств вычислительной техники (СВТ): всё более сложные процедуры передаются от человека к ЭВМ. Цель развития СВТ — повышение производительности интеллектуального труда за счёт увеличения доли машинного труда в общем процессе постановки и решения задач, то есть повышение интеллектуальных способностей ЭВМ.

Особенности традиционных и интеллектуальных ЭВМ

ЭВМ первого, второго и третьего поколений в основном построены по концепции **Дж. фон Неймана**. Её основные принципы:

1. **Принцип двоичного кодирования** — ЭВМ работает в двоичной, а не десятичной системе счисления.
2. **Принцип программного управления** — машина выполняет вычисления по программе; программа состоит из команд, выполняемых автоматически друг за другом в определённой последовательности.
3. **Принцип хранимой программы** — программа размещается в запоминающем устройстве с высокой скоростью выборки и записи.
4. **Принцип однотипности представления чисел и команд** — команды, как и числа, записываются в двоичном коде; это позволяет машине выполнять операции над командами программы.
5. **Принцип иерархичности памяти** — память строится иерархически, минимум из двух уровней: основная и внешняя память.
6. **Принцип адресности основной памяти** — память состоит из пронумерованных ячеек, доступных программе по двоичному адресу или присвоенному имени.

Структура ЭВМ по фон Нейману содержит: **управляющее устройство, арифметическое устройство, основную (оперативную) память, устройство ввода программ и данных, устройство вывода результатов, пульт управления.**

Такие ЭВМ называют **традиционными**. Они имеют **низкий интеллектуальный уровень**, так как требуют сравнительно большого участия человека в вычислительном процессе. ЭВМ четвёртого поколения обладают большими функциональными возможностями и более высоким интеллектуальным уровнем; ЭВМ ещё не появившегося пятого поколения должны были стать **интеллектуальными системами (ИС)**.

Общие понятия об интеллектуальных системах. Виды знаний. Функциональные механизмы

Понятие интеллектуальной системы возникло как обобщение моделей поведения биологических и искусственных систем, проявляющих способность к активному восприятию и избирательному запоминанию информации с целью формирования модели собственного поведения (или принятия решения).

Любая ИС должна обладать набором **интеллектуальных качеств**:

- свобода выбора цели и способа её достижения;
- возможность корректировать поведение при изменении обстоятельств;
- способность сохранять функциональные свойства при изменении окружающей среды.

Функциональные средства ИС предполагают наличие **функциональных механизмов (процессоров) и механизмов памяти (баз знаний)**.

Знания — любая информация, хранящаяся в системе и используемая ею для выполнения функционального назначения; по сути, это структурированные данные и процедурно заданные способы (алгоритмы) поведения системы. Уровень интеллекта во многом определяется содержательностью знаний. Знания разделяются на:

- **интерфейсные** — знания о принципах взаимодействия ЭВМ с внешней средой: о языках общения (лексика, синтаксис), способах коммуникации (диалог), формах представления информации, средствах общения;
- **прикладные** — формализованные знания, содержащие смысловые понятия, отношения, образы поведения и способы формирования новых знаний (процедурные знания об умениях, а также проблемные данные о внешней среде); во многом определяют интеллектуальную способность системы;
- **системные** — знания о самой системе (ресурсы, состояние, способы доступа, алгоритмы управления, контроля и т. д.).

Функциональные механизмы (процессоры) включают подсистемы **общения, логического вывода, решения задач**.

Эволюция интеллектуальных свойств ЭВМ

Процесс эволюции ЭВМ можно представить как повышение уровня её интеллекта. **Особенность интеллектуальной системы** (как человеко-машинной) — участие человека в процессе переработки информации.

Уровень интеллектуальности системы определяется отношением времени решения задачи человеком ко времени решения той же задачи в человеко-машинной системе. В оценке участвуют:

- **τ** — показатель относительной производительности ЭВМ (отношение времени решения типовой задачи человеком ко времени решения её ЭВМ);
- **α** — вес машинных операций в общем процессе решения задачи.

Важный вывод: **только рост производительности ЭВМ (τ) не приводит к значительному росту показателя интеллектуальности**, если α малó. С развитием ЭВМ величина α постоянно увеличивалась — менялось разделение труда между человеком и машиной.

В лекции приведена классификация поколений ЭВМ по свойствам, определяющим уровень их интеллекта (фрагменты таблицы; «1» — свойство присуще поколению, «0» — нет, «х» — не является определяющим):

Операционная среда (по поколениям 1–6): реальная машина → процедурная → виртуальная → объектная → интеллектуальная машина → искусственный «разум».

Виды знаний: процедурные (с 1-го поколения) → интерфейсные (со 2-го) → системные (с 3-го) → проблемные (с 4-го) → общие (с 5-го) → мета (с 6-го).

Языки общения: машинные → процедурные → диалоговые → функциональные → прикладные → естественные (по мере роста поколения).

Исторический факт из лекции: начиная с 60-х гг. XX века Минск был одним из ведущих центров СССР по разработке и производству вычислительной техники — более 60 % всех ЭВМ, применявшихся тогда в республиках СССР, разработаны и изготовлены в Минске.

Тема 2. Информационно-логические основы построения ЭВМ

Системы счисления, применяемые в вычислительной технике

Системы счисления делятся на **непозиционные** и **позиционные**. В вычислительной технике применяются позиционные системы:

- двоичная, восьмеричная, шестнадцатеричная, а также двоично-десятичная система счисления.

Формы представления информации в ЭВМ. Представление чисел

Числа в ЭВМ представляются:

- в естественной форме — с фиксированной точкой (ФТ);
- в нормальной форме — с плавающей точкой (ПТ).

Сравнение точности форм ФТ и ПТ (по материалам лекции):

Вид ошибки	Форма с фиксированной точкой (ФТ)	Форма с плавающей точкой (ПТ)
Абсолютная ошибка	Постоянна (const): для дробных чисел $\frac{1}{2} \cdot 2^{-n}$, для целых $\frac{1}{2}$	Переменна (var): $\frac{1}{2} \cdot 2^{-n} \cdot 2^p$, изменяется в зависимости от порядка p
Относительная ошибка	Переменна : от $\approx 2^{-(n+1)}$ до $\frac{1}{2} = 50\%$	Практически постоянна: $\approx 2^{-n}$

где n — количество значащих цифр в мантиссе, p — порядок числа.

Вывод: у формы ФТ абсолютная ошибка постоянна, но относительная сильно меняется; у формы ПТ, наоборот, относительная ошибка примерно постоянна, а абсолютная варьируется.

Машинный код произведения/частного при операциях масштабирования: для сложения и вычитания требуется равенство масштабных коэффициентов $m_3 = m_1 = m_2$; при умножении $m_3 = m_1 \cdot m_2$; при делении $m_3 = m_1 / m_2$.

Единицы измерения информации: бит (1 разряд), байт (8), параграф (16, в ПК), Кбайт = 2^{10} , Мбайт = 2^{20} , Гбайт = 2^{30} , Тбайт = 2^{40} , Пбайт = 2^{50} .

Длины полей (различаются в ЭВМ общего назначения и в ПК):

Длина	В ЭВМ общего назначения	В ПК
1 байт	байт	полуслово
2 байта	полуслово	слово
4 байта	слово	двойное слово
8 байт	двойное слово	расширенное слово
16 байт	учетверённое слово	—

Кодирование чисел в ЭВМ (прямой, обратный, дополнительный коды)

Знак числа x	Прямой код $[x]_{пр}$	Дополнительный код $[x]_{доп}$	Обратный код $[x]_{обр}$
$x \geq 0$	x	x	x
$x < 0$	$\sim 10^N +$	x	$\sim$

Для неотрицательных чисел все три кода совпадают; различия проявляются только для отрицательных чисел.

Логические переменные и функции. Функционально полные наборы

Над логическими переменными определяются логические функции (ЛФ). **Основной функционально полный набор (ОФПН)** образуют три элементарные функции:

Отрицание (НЕ) — логическая инверсия. Обозначение $y = \bar{x} = \neg x$.

x	0	1
y	1	0

Свойства: двойное отрицание $\bar{\bar{x}} = x$; если $f_1 = f_2$, то $\bar{f}_1 = \bar{f}_2$.

Конъюнкция (И) — логическое умножение. $y = x_1 \& x_2 = x_1 \cdot x_2 = x_1 \wedge x_2$.

x ₁	0	0	1	1
x ₂	0	1	0	1
y	0	0	0	1

Свойства: $x \cdot x \cdot \dots \cdot x = x$; $1 \cdot x = x$; $0 \cdot x = 0$; $x_i \cdot \bar{x}_i = 0$.

Дизъюнкция (ИЛИ) — логическое сложение. $y = x_1 + x_2 = x_1 \vee x_2$.

x ₁	0	0	1	1
x ₂	0	1	0	1
y	0	1	1	1

Свойства: $x + x + \dots + x = x$; $0 + x = x$; $1 + x = 1$; $x_i + \bar{x}_i = 1$.

Законы и следствия (правила) алгебры логики

Основные законы (формулируются для дизъюнкции и конъюнкции):

- Переместительный (коммутативный):** $x_1 + x_2 = x_2 + x_1$; $x_1 \cdot x_2 = x_2 \cdot x_1$.
- Сочетательный (ассоциативный):** $x_1 + x_2 + x_3 = x_1 + (x_2 + x_3) = \dots$; аналогично для произведения.
- Распределительный (дистрибутивный):**
 - закон 1-го рода: $(x_2 + x_3) \cdot x_1 = x_1 \cdot x_2 + x_1 \cdot x_3$;
 - закон 2-го рода: $x_2 \cdot x_3 + x_1 = (x_1 + x_2) \cdot (x_1 + x_3)$. Закон дистрибутивности дизъюнкции относительно конъюнкции аналога в обычной алгебре не имеет — справедлив только в логике.
- Закон инверсии (правило де Моргана):** отрицание суммы равно произведению отрицаний $x_1 + x_2 + \dots + x_n = \bar{x}_1 \cdot \bar{x}_2 \cdot \dots \cdot \bar{x}_n$; отрицание произведения равно сумме отрицаний $x_1 \cdot x_2 \cdot \dots \cdot x_n = \bar{x}_1 + \bar{x}_2 + \dots + \bar{x}_n$.

Следствия (правила) из законов алгебры логики:

- Правило старшинства** — порядок выполнения функций по ступеням: 1-я ступень — НЕ, 2-я — И, 3-я — ИЛИ. Действия одной ступени выполняются слева направо; при разных ступенях — сначала 1-я, затем 2-я, затем 3-я.
- Правило склеивания** (для соседних элементарных функций) — логическую сумму двух соседних произведений ранга r можно заменить элементарным произведением ранга $r-1$ (общей частью слагаемых); двойственно для конъюнкции (через суммы).
- Правило поглощения** (для функций разного ранга, меньшая — часть большей) — логическую сумму двух произведений разных рангов, одно из которых часть другого, можно заменить слагаемым меньшего ранга; двойственно для конъюнкции.
- Правило развёртывания** произвольной функции в совокупность конституент «1» и «0» — выполняется в 3 этапа (введение недостающих переменных через $x_i \cdot \bar{x}_i$ или $x_i + \bar{x}_i$ и раскрытие по распределительному закону), что разворачивает элементарную функцию ранга r в 2^{n-r} конституент.

Этапы логического проектирования устройств. Синтез комбинационных схем

Комбинационным устройством (КУ) называют устройство **без памяти**, в котором выходные сигналы в любой момент определяются **только комбинацией входных сигналов**.

Синтез КУ выполняется в **4 этапа**:

1. составление **таблицы истинности** синтезируемого КУ согласно его назначению и описанию работы;
2. составление **математических формул** для ЛФ, описывающих работу КУ по таблице истинности;
3. **анализ и оптимизация (минимизация)** полученных функций — построение различных вариантов выражения на основе законов алгебры логики и выбор наилучшего по заданному критерию;
4. составление **функциональной (логической) схемы** КУ из элементов НЕ, И, ИЛИ (прямое замещение элементарных произведений, сумм и отрицаний конъюнкторами, дизъюнкторами и инверторами).

Классический пример из лекции — **синтез полусумматора**: два входа (цифры слагаемых x_1, x_2) и два выхода (цифра суммы S , цифра переноса P).

Нормальные формы представления логических функций (СДНФ, СКНФ)

- **Совершенная дизъюнктивная нормальная форма (СДНФ)**. Правило 1: чтобы получить СДНФ функции, заданной таблицей, составляют **сумму конститuent единицы** для всех наборов, на которых $f_j = 1$; символ переменной берётся с отрицанием, если её значение в наборе равно 0.
- **Совершенная конъюнктивная нормальная форма (СКНФ)**. Правило 2: составляют **произведение конститuent нуля** для всех наборов, на которых $f_j = 0$; символ переменной берётся с отрицанием, если её значение в наборе равно 1.

Основные методы минимизации логических функций

Известны три метода:

1. **расчётный** — метод непосредственных преобразований;
2. **расчётно-табличный** — метод Квайна – Мак-Класки;
3. **табличный** — метод Вейча – Карно.

Минимизация в общем случае идёт в **три этапа**:

1. переход от СДНФ (СКНФ) к **сокращённой** форме путём всех возможных склеиваний (сначала конститuent, потом импликант более низкого ранга);
2. переход от сокращённой к **тупиковой** форме (тупиковая — ДНФ/КНФ, члены которой — простые импликанты, среди которых нет ни одной лишней);
3. переход от тупиковой к **минимальной** форме (факторизация) — этап нерегулярный, требует опыта и интуиции.

Расчётный метод: лишней считается импликанта, которая не влияет на значение истинности (для ДНФ — на наборе, где она равна 1, сумма остальных членов тоже равна 1; для КНФ двойственно — через 0).

Расчётно-табличный метод (Квайна – Мак-Класки): этапы 1 и 3 как в расчётном, а тупиковая форма находится с помощью специальной таблицы (столбцы — конститuent, строки — импликанты сокращённой формы); если импликанта — часть конститuent, ставится знак «X»; задача — вычеркиванием лишних импликант оставить минимум знаков «X» в каждом столбце. Для машинной обработки Мак-Класки предложил **числовое представление ЛФ** (десятичный индекс реализации, развёрнутый в n -разрядное двоичное число, восстанавливает набор аргументов).

Табличный метод (Вейча – Карно): первые два этапа выполняются с помощью **диаграммы Вейча – Карно** — прямоугольника из 2^n клеток; каждая клетка соответствует одной конститuent. Обязательное **условие совмещённого соседства** (в соседних клетках — соседние конститuent) обеспечивается нумерацией координат в **циклическом коде Грея** (два числа, отличающиеся на единицу, различаются только в одном разряде). Правила: 2^1 смежных клеток в форме прямоугольника соответствуют одной конъюнкции (дизъюнкции) ранга $r = n - i$; соседними считаются также клетки на противоположных концах любой строки и столбца.

Классификация логических функций. Функциональная полнота наборов

Под **элементарными** понимают ЛФ одного и двух аргументов. При двух аргументах существует $N = 2^4 = 16$ различных двухэлементных ЛФ ($f_0 \dots f_{15}$).

Для подбора функций в функционально полные наборы (ФПН) функции классифицируют по свойствам: **линейные, сохраняющие «0», сохраняющие «1», самодвойственные, монотонные.**

- **Сохраняющие 0:** $f(0,0,\dots,0) = 0$ — первые 8 функций $f_0 \dots f_7$.
- **Сохраняющие 1:** $f(1,1,\dots,1) = 1$ — нечётные функции $f_1, f_3, f_5, f_7, f_9, f_{11}, f_{13}, f_{15}$.
- **Монотонные:** при возрастании набора аргументов значение функции не убывает — 6 функций $f_0, f_1, f_3, f_5, f_7, f_{15}$ (проверка по путям $f(0,0) \Rightarrow f(0,1) \Rightarrow f(1,1)$ или $f(0,0) \Rightarrow f(1,0) \Rightarrow f(1,1)$).

Теорема Глушкова о функциональной полноте. Чтобы набор ЛФ был функционально полным, необходимо и достаточно, чтобы в него входили:

- хотя бы одна **нелинейная** функция;
- хотя бы одна **не сохраняющая «1»**;
- хотя бы одна **не сохраняющая «0»**;
- хотя бы одна **не самодвойственная**;
- хотя бы одна **не монотонная**.

Логические базисы и поколения ЭВМ. По реализуемости техническими средствами ЛФ делят на 3 группы: непосредственно реализуемые на современных переключательных устройствах; реализуемые только на некоторых устройствах; косвенно реализуемые через функции первых двух групп. В ЭВМ 1-го и 2-го поколений использовался базис **НЕ-И-ИЛИ**; с появлением интегральных схем (3-е поколение) стали применять базисы **Шеффера (И-НЕ)** и **Пирса (ИЛИ-НЕ)**. Операцию Шеффера трактуют как «шефферово умножение»: отрицание аргумента — это «шефферов квадрат» $\bar{x}_i = x_i \cdot x_i = x_i / x_i$. К шефферовым произведениям применимы правила склеивания и поглощения; синтез в базисе Шеффера тоже выполняется в 4 этапа.

Элементы памяти (триггеры). Построение регистров, сдвигателей, счётчиков

Для построения устройств **с памятью** кроме комбинационного ФПН используются **запоминающие элементы**. Основной элемент накапливающих (некомбинационных) схем — **триггер (Т)**: элемент с двумя устойчивыми состояниями; переход из одного состояния в другое происходит скачкообразно с изменением уровня выходного напряжения.

Поведение триггеров описывается **таблицами переходов**. Основные типы: **R-S, T, D, J-K**. Обозначения R-S-триггера: S — установка в 1, R — установка в 0, с — вход синхронизации, Q — единичный выход, $\bar{Q}$ — нулевой выход. **J-K-триггер** универсален — на его основе строятся триггеры других типов (D, T, R-S).

С помощью триггеров строятся **функциональные устройства**: регистры (в т. ч. с предварительным сбросом), сдвигающие регистры (сдвигатели), счётчики (например, суммирующий 4-разрядный счётчик), а также широко применяются **дешифраторы**.

Синтез схем с памятью (цифровых автоматов)

Цифровой автомат (ЦА) — устройство, перерабатывающее слова входного алфавита X в слова выходного алфавита Y; состояние меняется скачкообразно, работа идёт по тактам (автоматное время). Закон функционирования:



$$S(t) = f[S(t-1), X(t-1)]; \quad (\text{функция переходов})$$
$$Y(t) = \varphi[S(t), X(t)]. \quad (\text{функция выходов})$$

- ЦА, задаваемый таблицами переходов и выходов в таком виде, — **автомат Мили**.

- Если выход зависит только от внутреннего состояния $Y(t) = \phi[S(t)]$ — это **автомат Мура** (для него строят отмеченную таблицу переходов).

Комбинационные устройства — частный случай ЦА без памяти.

Условие функциональной полноты для синтеза ЦА: система должна содержать (а) логические элементы, образующие ФПН для синтеза комбинационной схемы, и (б) запоминающие элементы с полной системой переходов и выходов. **Триггер** — простейший автомат Мура с полной системой переходов и выходов, поэтому любой комбинационный ФПН, дополненный триггером, функционально полон для синтеза ЦА с памятью (вместо триггера можно использовать линии задержки).

Канонический метод структурного синтеза ЦА (В. М. Глушков). Исходные данные: словесное описание автомата, таблицы переходов элемента памяти, логический базис. Автомат разделяется на **комбинационную схему** и **память**; синтез ЦА сводится к синтезу комбинационной схемы. Процесс — в **4 этапа**:

1. переход от словесного описания к таблицам переходов и выходов и от абстрактного автомата к структурному; расчёт числа входов $e \geq \log_b k$, элементов памяти $n \geq \log_b l$, выходов $w \geq \log_b m$;
2. раскрытие структуры памяти; подсчёт связей $p = e_{\text{зн}} \cdot n$; комбинационная схема оформляется как «чёрный ящик» с $e_{\text{общ}} = e + n$ входами и $w_{\text{общ}} = w + p$ выходами;
3. построение **таблицы возбуждения памяти** на основе таблиц переходов автомата и элемента памяти (сигналы возбуждения $h_j = \Psi(v_1, \dots, v_e)$);
4. **комбинационный синтез** — логический синтез комбинационной схемы по таблице выходов ЦА и таблице возбуждения памяти, сводимым в единую таблицу истинности.

Тема 3. Принципы построения и функционирования ЭВМ

Основные понятия о принципах организации ЭВМ

- **Система** — множество элементов в отношениях и связях друг с другом, образующее целостность.
- **Функция** — правило получения результатов, определяемых назначением системы (описание процессов; формы описания — словесная, формулы, алгоритмы).
- **Структура** — совокупность элементов и связей между ними (математическая форма — граф; инженерная — схема).

Свойство совокупности элементов, объединённых в систему, **не является суммой свойств элементов**, а имеет новое качество. Принцип, по которому объединение элементов даёт новое качество, называется **принципом организации**.

- **Функциональная организация** — принципы построения абстрактных структур (порождение функций).
- **Структурная организация** — принципы перевода абстрактных систем (заданных функциями) в материальные системы из физических элементов.

ЭВМ — искусственная система для выполнения вычислений на основе алгоритмов, поэтому **свойства алгоритмов определяют организацию ЭВМ**. Важнейшие свойства алгоритмов: **дискретность** информации; **конечность** и **элементарность** набора операций; **детерминированность** вычислительных процессов.

Принцип программного управления. Концепция фон Неймана

Современные ЭВМ строят по **принципу программного управления**: алгоритм представляется в форме **операторной схемы**, задающей вычисления как композицию операторов (преобразования информации и анализа для определения порядка выполнения). Один из способов реализации этого принципа предложен в **1945 г. Дж. фон Нейманом** (основные принципы фон Неймана см. в Теме 1).

Состав и порядок функционирования традиционных ЭВМ

Номенклатура устройств ЭВМ вытекает из фон-неймановской концепции. В состав традиционной ЭВМ входят подсистемы: обрабатывающая, памяти, ввода-вывода и обслуживания (реализация подсистем в больших, малых, микро-ЭВМ различна).

Состав ПК. Центральное устройство — **микропроцессор (МП)**, управляющий работой всех блоков и выполняющий арифметические и логические операции. В состав МП входят:

- **Устройство управления (УУ)** — формирует и подаёт сигналы управления, формирует адреса ячеек памяти; опорную последовательность получает от **генератора тактовых импульсов**;
- **Арифметико-логическое устройство (АЛУ)** — выполняет арифметические и логические операции (для ускорения может подключаться математический сопроцессор);
- **Интерфейсная система МП** — сопряжение и связь с другими устройствами (внутренний интерфейс, буферные регистры, схемы управления портами ввода-вывода и системной шиной);
- **Микропроцессорная память (МПП)** — кратковременное хранение информации ближайших тактов; строится на **регистрах** (быстродействующие ячейки различной длины, в отличие от ячеек ОП стандартной длины).

Интерфейс — совокупность средств сопряжения и связи устройств. **Порты ввода-вывода** — элементы системного интерфейса для обмена МП с другими устройствами. **Системная (общая) шина** — основная интерфейсная система ПК; включает кодовые шины адреса (КША), данных (КШД) и инструкций (КШИ), а также шину питания.

Порядок функционирования ЭВМ: инициализация ЭВМ → ввод программы и данных в память → запись программы → выполнение программы → вывод результатов.

Функциональная организация ЭВМ (архитектура). Абстрактные элементы

С функциональной точки зрения ЭВМ — это **система абстрактных элементов**, в терминах которых представляется информация (данные и алгоритмы) и процессы вычислений. К таким элементам относятся:

- наборы символов и машинные элементы информации;
- машинные операции и команды;
- адреса и способы адресации;
- управляющие слова (слово состояния программы — ССП, и др.).

Структура ЭВМ — состав элементов и конфигурация связей, определяемые функциями. При одинаковых функциях структуры могут отличаться (ЭВМ общего назначения и ПК). **Функции первичны по отношению к структуре**, поэтому проектирование начинается с конкретизации функций (принципа программного управления). Для круга вопросов организации ЭВМ часто применяют термин **«архитектура»**.

1. Машинные элементы информации: двоичные коды; байты, слова; представление чисел в формах ФТ, ПТ, двоично-десятичной, символьной.

2. Машинная операция — действие, инициируемое одной командой и реализуемое оборудованием. С набором операций связана **система команд**, в которую входят команды: пересылки информации внутри компьютера; арифметические и логические; операции над строками; обращения к внешним устройствам; передачи управления (условные и безусловные переходы); обслуживания и вспомогательные.

3. Способы адресации памяти:

- **непосредственная** — в команде указано само значение операнда, а не адрес;
- **прямая** — указан абсолютный или исполнительный адрес операнда;
- **косвенная** — указан регистр или ячейка, где находится адрес операнда;
- **ассоциативная** — указан содержательный признак операнда (в ассоциативных ЗУ);
- **неявная** — адрес не указан, но подразумевается кодом операции.

В ПК также применяются **относительная** и **стековая** адресации.

Режимы работы ЭВМ

Основные режимы: **однопрограммный, многопрограммный, пакетный, реального времени** и др. Наиболее сложным является **многопрограммный** режим.

Требования и средства для многопрограммного режима

Для обеспечения многопрограммного режима в ЭВМ должны быть выполнены требования:

- наличие средств, управляющих **порядком выполнения задач**;
- **ёмкость памяти**, достаточная для размещения информации нескольких задач;
- **защита памяти**;
- **система прерывания** программ (для сообщения об особых ситуациях);
- возможность **параллельной работы** процессора и системы ввода-вывода.

Управление порядком задач обычно реализуется программными средствами; принципы реализации остальных требований рассматриваются в подсистемах ЭВМ (см. ниже).

Построение и функционирование основных подсистем традиционных ЭВМ

Обрабатывающая подсистема. В любом устройстве обработки цифровой информации выделяют **операционный блок (ОБ)** и **управляющий блок (УБ)**. В ПК функции обрабатывающей подсистемы выполняет МП, разделяемый на операционную часть (УУ, АЛУ, МПП) и интерфейсную (адресные регистры МПП, блок регистров команд, схемы управления шинами и портами); обе части работают параллельно.

Прерывания. При возникновении событий, требующих немедленной реакции, ЭВМ прерывает текущую программу и переходит к специальной программе. **Система прерываний** выполняет функции: запоминание состояния прерываемой программы в стеке; приоритетный выбор запроса и переход к прерывающей программе; восстановление состояния и возврат; программное изменение приоритетов (маскирование). Распространённый способ организации — **слова состояния программы (ССП)**; пример полей СПП — АМНР (А — режим ASCII, М — маска контроля машины, W — состояние ожидания, R — режим решения основной задачи).

Подсистема памяти. Важнейшие характеристики ЗУ: **ёмкость, удельная ёмкость, быстродействие**. Требования к ёмкости и быстродействию противоречивы, поэтому память организуется **иерархически**. В ПК три основных уровня — **МПП, основная память (ОП), внешняя память (ВЗУ)** — плюс промежуточная буферная (**КЭШ**) память:

Тип памяти	Ёмкость	Быстродействие
МПП	десятки байт	$t_{\text{обр}} = 1\text{--}10 \text{ нс}$
ОП: ОЗУ	гигабайты	$t_{\text{обр}} = 50\text{--}100 \text{ нс}$
ОП: ПЗУ	мегабайты	$t_{\text{обр}} = 30\text{--}200 \text{ нс}$
ВЗУ: НМД	терабайты	$t_{\text{дост}} = 5\text{--}30 \text{ мс}; V_{\text{чит}} = 500\text{--}5000 \text{ Кбайт/с}$
ВЗУ: CD/DVD	тысячи Мбайт	$t_{\text{дост}} = 50\text{--}100 \text{ мс}; V_{\text{чит}} = 300\text{--}5000 \text{ Кбайт/с}$

По методам размещения и поиска информации различают память: **адресную** (поиск по номеру/адресу ячейки), **ассоциативную** (поиск по содержанию — ассоциативному признаку, параллельно во всех ячейках), **стековую** и **магазинную**.

Организация памяти в многопрограммных системах. Так как разместить все программы в ОП невозможно, используют ВЗУ большого объёма; целевые программы не привязывают к конкретным адресам ОП. При подготовке используются **условные адреса**, которые супервизор переводит в **исполнительные** способами: **базирование (фрагментация)** и **организация виртуальной памяти**. **Виртуальная память (ВП)** обеспечивает динамическое распределение, устраняет фрагментацию; программист работает с «кажущейся» памятью объёмом, равным адресному пространству; преобразование виртуальных адресов в физические выполняется автоматически при выполнении команды (упрощается при **страничной** структуре памяти).

Защита памяти. Средства защиты проверяют корректность адреса при каждом обращении; некорректное обращение блокирует программу (прерывание по нарушению защиты). Способы: по **граничным адресам**

(неудобен — требует сплошного участка ОП), по **ключам** и др. Режимы защиты: только по записи; по любому обращению.

Подсистема ввода-вывода. Для параллельной работы процессора и ввода-вывода используются управляющие слова. Уровни (устройства): **каналы ввода-вывода; устройства управления внешними устройствами (в ПК — адаптеры); внешние устройства (механизмы).** Управляющие слова: адресное слово канала (АСК), командное слово канала (КСК), управляющее слово устройства (УСУ), слово состояния канала (ССК). В ПК применяются адаптеры на системной шине, **контроллер прямого доступа к памяти (DMA)** — обмен ВнУ ↔ ОП без участия МП, и **сопроцессор ввода-вывода.**

Тема 4. Развитие структуры и архитектуры ЭВМ

Основные тенденции и направления развития структуры ЭВМ

Развитие архитектуры неизбежно ведёт к развитию структуры. **Условие и критерий развития структуры — рост производительности;** эффективность структуры определяется отношением **стоимость/производительность.**

Требования к ЭВМ разной производительности различны:

- **ЭВМ малой производительности** (массовые) — важна рентабельность (минимизация стоимости, габаритов, эксплуатационных расходов); структура на базе универсальных устройств (микропроцессоры, память, адаптеры);
- **ЭВМ средней производительности** — дополнительное специализированное оборудование, более высокая разрядность блоков и шин;
- **ЭВМ высокой производительности** — максимальная производительность и точность; все методы структурной оптимизации, высокая специализация, повышенная разрядность (64–128 бит).

Основные тенденции развития структуры традиционных ЭВМ — разделение функций системы и максимальная специализация подсистем для выполнения этих функций.

Направления развития основных подсистем

Обрабатывающая подсистема. Развивается более всех остальных по пути разделения функций и специализации. В ПК её функции выполняет **микропроцессор (МП, CPU)** — функционально законченное программно-управляемое устройство на одной или нескольких БИС/СБИС. Функции МП: вычисление адресов команд и операндов; выборка и дешифрация команд; выборка данных из ОП, МПП, регистров адаптеров; приём/обработка запросов от адаптеров; обработка и запись данных; выработка управляющих сигналов; переход к следующей команде. Основные параметры: разрядность, тактовая частота, виды и размер КЭШ-памяти, состав инструкций, конструктив, энергопотребление, рабочее напряжение. Группы МП:

- **CISC** (Complex Instruction Set Command) — полный набор команд;
- **RISC** (Reduced Instruction Set Command) — усеченный набор;
- **VLIW** (Very Long Instruction Word) — сверхдлинное командное слово;
- **MISC** (Minimum Instruction Set Command) — минимальный набор и высокое быстродействие.

Современные технологии: Hyper-Threading (многопоточность), гиперконвейерная обработка, рост частоты системной шины, КЭШ с отслеживанием выполнения команд, расширенные операции с плавающей точкой и мультимедийные, потоковые SIMD-расширения, RAID; многоядерность.

Подсистема внутренней памяти. Развитие идёт в направлении увеличения **объёма и пропускной способности.** Основа ОЗУ ПК — микросхемы динамической памяти **DRAM** (матрицы МОП-транзисторов). Конструктивно — модули памяти (DIP, SIP, SIPP, SIMM, DIMM, RIMM). Типы ОЗУ: FRAM, DRAM, EDO, BEDO, SDRAM, DDR SDRAM, DRDRAM и др. Новые типы памяти: **FeRAM** (ферроэлектрическая энергонезависимая), **MRAM** (магнитная), **PCM** (с изменением фаз), **PMC** (на программируемых металлизированных ячейках), молекулярная память, **NRAM** (на углеродных нанотрубках).

Подсистема ввода-вывода. Направления: повышение пропускной способности, расширение функций, повышение независимости подсистемы. Основной способ повышения пропускной способности в ЭВМ общего назначения (mainframe) — увеличение числа **каналов** (32–256 каналов, до 256 ВнУ в каждом). Типы интерфейса ввода-вывода: параллельный ($V = 4,5$ Мб/с, $L = 60$ м), **ESCON** (оптоволоконный, 17 Мб/с, несколько км), **FICON** (100 Мб/с, сотни км). В ПК — переход к **локальным шинам**, подключаемым к шине МП: VLB (VESA Local Bus), ISA, MCA, EISA, PCI, AGP.

Подсистема управления и обслуживания (ПУО). Выделение ПУО в самостоятельную единицу — логическое развитие принципа усложнения ЭВМ. Это аппаратно-программный комплекс; задачи: инициализация ЭВМ; поддержка ручных операций оператора; автоматическая диагностика сбоев и отказов; накопление статистики о вычислительном процессе. Основное направление развития — **увеличение уровня автоматизации управления.**

Развитие операционных сред (архитектур) в ЭВМ разных поколений

Операционная среда — комплекс программного обеспечения, предоставляющий средства разработки и выполнения прикладных программ (операционная система, интерфейсы прикладных программ, прикладные программы, сетевые службы, базы данных, языки). С развитием поколений ЭВМ операционная среда изменялась.

Концептуальные модели виртуальных, объектно-ориентированных и интеллектуальных ЭВМ

Виртуальная ЭВМ (VM). Основная концепция ЭВМ 3-го поколения — совместное использование (разделение) ресурсов многими пользователями. Дальнейшее развитие — принцип **виртуального распределения ресурсов**. Основное свойство архитектуры VM — **отсутствие в интерфейсе пользователя ограничений на ресурсы**. Достоинства: одно задание выполняется на ЭВМ с разными конфигурациями; возможны развитие ЭВМ и адаптация новых технических средств; каждая VM надёжна и защищена от вмешательства извне. Недостаток — большие системные потери на организацию виртуальных средств.

Объектная (объектно-ориентированная) ЭВМ (OM). Развитие архитектуры VM; характерное свойство — операционная среда **более высокого уровня**, определяемая в терминах **процессов и данных**, что полностью исключает из интерфейса пользователя понятие реальной машины. Концепция предусматривает:

1. высокоуровневый **объектный машинный язык (ОМЯ)** с единым представлением всех объектов задачи и командами высокого уровня;
2. **одноуровневую память** с практически неограниченным адресным пространством и уникальными идентификаторами объектов;
3. интегрированные в язык средства управления процессами и данными (в т. ч. организации БД), упрощающие программирование;
4. интегрированные в аппаратуру средства управления ресурсами физического уровня, упрощающие функции ОС.

Архитектура OM ещё более приспособлена к будущим эволюциям; гибкость обеспечивается микропрограммным управлением; снижается объём системного ПО, повышаются надёжность, защита, гибкость.

Интеллектуальная ЭВМ (5-е поколение). Естественное развитие ЭВМ в направлении повышения **доступности для конечного пользователя**. Возможности: **интеллектуальный интерфейс** (доступ на языке проблемной области в естественной форме — текст, речь); развитые инструментальные системы программирования; **база разнообразных знаний** с механизмами упорядочивания и поиска; средства распределённого доступа (сеть интеллектуальных персональных ЭВМ).

Подсистемы ЭВМ 5-го поколения:

1. **Подсистема общения** с пользователем (интеллектуальный интерфейс): процессоры речевого и визуального ввода, коммуникационный процессор, процессор управления объектами;
2. **Подсистема анализа и логического вывода** — выбор методов решения и синтез программ; в её состав входят **лингвистический (языковой) процессор, символные процессоры (Лисп, Пролог), процессор распознавания;**

3. **Подсистема решения задач** — средства высокой производительности;
4. **Подсистема управления и обслуживания** (сервисная) — надёжность, перемещаемость программ и данных, контроль и восстановление;
5. **База знаний** — накопление и хранение интерфейсных, проблемных и системных знаний.

Языки функционального и логического программирования. В основе функциональных языков — понятие **функции** (математической), в основе логических — понятие **отношения** (предикатные логики). Функция $f: (x_1, \dots, x_n) \rightarrow y$ представима как отношение $R(x_1, \dots, x_n, y)$. **ЛИСП** предназначен для обработки списков, **Пролог** — для логического программирования; оба относятся к языкам 5-го поколения. Для обычных машин это языки высокого/сверхвысокого уровня, а в **логических и функциональных машинах** нового типа они являются машинными языками (Лисп- и Пролог-машины, объектно-ориентированная Смолток-машина «Катана»). На современном уровне эти машины ещё последовательные, но языки логического и функционального типов будут применяться и в машинах **параллельного** типа.

Тема 5. Параллельные компьютеры для интеллектуальных систем

Основные особенности и требования к интеллектуальным системам обработки знаний

Интеллектуальные системы (поддержки принятия решений, экспертные, обучающие, тренажёрные и др.) в своей основе — **системы обработки знаний**. Их специфика обусловлена:

1. **большими объёмами** перерабатываемой информации;
2. **высоким уровнем сложности структур** перерабатываемых данных;
3. **трудно формализуемым характером** перерабатываемых знаний.

Практическая ценность ИС (особенно для принятия решений в чрезвычайных ситуациях и управления реальными объектами) определяется работой в **реальном масштабе времени**, что трудно достижимо без **параллельной обработки** и **ассоциативного доступа** к структурам данных. Из-за трудоёмкости разработки жизненный цикл таких систем должен быть длительным (десяток лет и более), тогда как сменяемость аппаратных и программных средств — от нескольких лет до месяцев. Поэтому ИС должны обладать качествами:

- **открытость** — модифицируемость, добавление новых методов представления и переработки знаний;
- **интегрируемость** — возможность работы в составе комплексов различных вычислительных и исполнительных средств.

Ни одна из существующих архитектур не обладает всеми этими свойствами в совокупности.

Параллелизм и его формы

Параллелизм — возможность одновременного выполнения нескольких арифметико-логических или служебных операций. На стадии постановки задачи параллелизм не определён — он появляется после выбора метода вычислений. Формы:

- **естественный (векторный)** параллелизм;
- параллелизм **независимых ветвей**;
- параллелизм **смежных операций (скалярный)**.

Скалярный параллелизм называют **мелкозернистым (МЗП)**; векторный параллелизм и параллелизм независимых ветвей — **крупнозернистым (КЗП)**, оперирующим крупными объектами (ветвями программ и векторами).

Оценка эффективности. Ускорение $r = T_1/T_n$, где T_1 — время решения на однопроцессорной системе, T_n — на n -процессорной. **Закон Амдала:** ускорение зависит и от потенциального параллелизма задачи (величины $1 - a$), и от числа процессоров n ; **предельное ускорение определяется свойствами задачи**. Например, при $a = 0,2$ ускорение не может превзойти 5 при любом числе процессоров; при 95 % параллельных вычислений максимальное ускорение — 20 независимо от числа процессоров.

Общая классификация вычислительных систем

Классификация М. Флинна (по наличию параллелизма в потоках команд и данных)

Класс	Расшифровка	Описание
ОКОД / SISD	Single Instruction – Single Data	Последовательные ЭВМ: одна программа, один счётчик команд, одно АЛУ
ОКМД / SIMD	Single Instruction – Multiple Data	Одна программа, каждая команда обрабатывает много чисел (векторный параллелизм)
МКОД / MISD	Multiple Instruction – Single Data	Несколько команд работают с одним элементом данных; на практике не применяется
МКМД / MIMD	Multiple Instruction – Multiple Data	Независимо выполняется несколько программных ветвей с обменом данными; многопроцессорные системы

Класс **МКМД (MIMD)** включает машины двух типов: с управлением от **потока команд** (IF — Instruction Flow, традиционное выполнение по порядку) и от **потока данных** (DF — Data Flow, активация операторов по мере готовности).

Классификация Головкина

Признаки, характеризующие организацию структуры и функционирования ВС:

1. **тип потока команд** в центральной части ВС (ОК — одиночный, МК — множественный);
2. **тип потока данных** (ОД — одиночный, МД — множественный);
3. **способ обработки данных** в центральных устройствах (С — пословная, Р — поразрядная);
4. **степень связанности** компонентов (Нс — низкая, Вс — высокая);
5. **степень однородности** основных компонентов (Ор — однородная, Нр — неоднородная);
6. **тип внутренних связей** в ВС:
 - по способу связи: **Кн** — «канал-канал», **Пм** — через общую внешнюю память, **Пр** — непосредственно между процессорами;
 - по структуре: **Ош** — через одну общую шину с разделением времени, **Мш** — через множество шин с многовходовыми модулями памяти, **Пк** — перекрёстные связи через матричный коммутатор.

Требования к схеме классификации ВС: возможность классификации всех существующих и предвидимых структур; дифференциация существенно различных структур; однозначность классификации любой ЭВМ/ВС.

Дополнительно в лекции упомянуты **Эрлангенская схема классификации** (триплет $t = (k, d, w)$: k — число устройств управления, d — число АЛУ на одно УУ, w — длина машинного слова; простая фон-неймановская ЭВМ — $t = (1, 1, w)$) и **классификация Энслоу** (по структуре аппаратных средств: МВС с общей/разделённой во времени шиной; с перекрёстной коммутацией; многоуровневые; матричные и векторные; ассоциативные; конвейерные).

Основные характеристики многопроцессорных систем

Основной параметр классификации параллельных компьютеров — наличие **общей** (как в SMP) или **распределённой** (как в MPP) памяти.

Массивно-параллельные системы (MPP). Однородные вычислительные узлы: один/несколько процессоров (обычно RISC), локальная память (прямой доступ к памяти других узлов невозможен), коммуникационный процессор/сетевой адаптер, иногда — устройства ввода-вывода; узлы связаны коммуникационной средой. ОС: либо полноценная ОС на управляющей машине (front-end) + урезанная на узлах (Cray T3E), либо полноценная UNIX-подобная ОС на каждом узле (IBM RS/6000 SP + AIX). Масштабируемость — до нескольких тысяч процессоров. Модель программирования — **передача сообщений** (MPI, PVM, BSPlib).

Симметричные мультипроцессорные системы (SMP). Несколько однородных процессоров и массив **общей памяти**; все процессоры имеют **равноправный доступ** к любой точке памяти с одинаковой скоростью (через общую шину или crossbar-коммутатор); аппаратно поддерживается **когерентность кэшей**. Общая память

упрощает взаимодействие, но ограничивает число процессоров (**не более 32** в реальных системах). Вся система — под управлением единой ОС. Модель программирования — **общая память** (POSIX threads, OpenMP); есть эффективные средства автоматического распараллеливания.

Системы с неоднородным доступом к памяти (NUMA). Однородные базовые модули (несколько процессоров + блок памяти), объединённые высокоскоростным коммутатором; **единое адресное пространство**; аппаратная поддержка доступа к удалённой памяти, но доступ к локальной в несколько раз быстрее. При аппаратной когерентности кэш — **cc-NUMA**. Масштабируемость ограничена объёмом адресного пространства и возможностями аппаратуры/ОС (до 256 процессоров — Origin2000). ОС обычно единая (как в SMP), модель программирования аналогична SMP. NUMA — нечто среднее между SMP и MPP: память физически распределена, но логически общедоступна; расширение SMP с более высоким уровнем масштабирования.

Параллельные векторные системы (PVP). Основной признак — **векторно-конвейерные процессоры** с командами однотипной обработки векторов, эффективно выполняемыми на конвейерных функциональных устройствах. Обычно несколько процессоров (1–16) работают с общей памятью (как SMP); узлы могут объединяться коммутатором (как MPP). Программирование — **векторизация циклов** и их распараллеливание.

Кластерные системы. Набор рабочих станций или ПК общего назначения как **дешёвый вариант MPP**; для связи узлов — стандартные сетевые технологии (Fast/Gigabit Ethernet, Myrinet) на базе шины или коммутатора. При объединении компьютеров разной мощности/архитектуры — **гетерогенные (неоднородные) кластеры**. Узлы могут одновременно использоваться как пользовательские рабочие станции. ОС — стандартные для рабочих станций, чаще свободно распространяемые (Linux/FreeBSD) со средствами параллельного программирования. Модель — **передача сообщений** (чаще MPI). Дешевизна оборачивается большими накладными расходами на взаимодействие процессов, что сужает класс решаемых задач.

Комбинированные архитектуры. Объединение архитектур для использования их преимуществ: MPP-системы, узлы (гиперузлы) которых — SMP (например, IBM RS/6000 SP); архитектура NUMA как распространение принципов SMP на MPP для преодоления ограничения масштабируемости (Exemplar SPP1200/SPP2000, Origin 2000, Sequent NUMA-Q, RM600E). Также упомянуты **метакомпьютеры** — распределённые вычисления в глобальных сетях (Интернет), которые, строго говоря, не являются параллельными архитектурами.

Применительно к машинам переработки знаний эти системы — аппаратная основа для реализации, например, СУБД и экспертных систем; при этом приложения изначально ориентируются на **распределённую параллельную реализацию** (особенно при использовании MPP и кластеров с большим числом процессоров).

Кластерные системы семейства «СКИФ»

В прочитанном конспекте лекций **отдельного раздела о кластерах семейства «СКИФ» нет**. «СКИФ» упоминается только в Теме 1 — в перечне характерных моделей/ПО ЭВМ **4-го поколения** (наряду с Cray и сетями), как разработки, связанные с белорусской (минской) школой вычислительной техники. Подробные характеристики семейства «СКИФ» в лекционном материале не приведены, поэтому здесь они не излагаются (чтобы не выходить за рамки конспекта). Общие принципы кластерных систем, к которым относится «СКИФ», изложены в предыдущем подразделе.